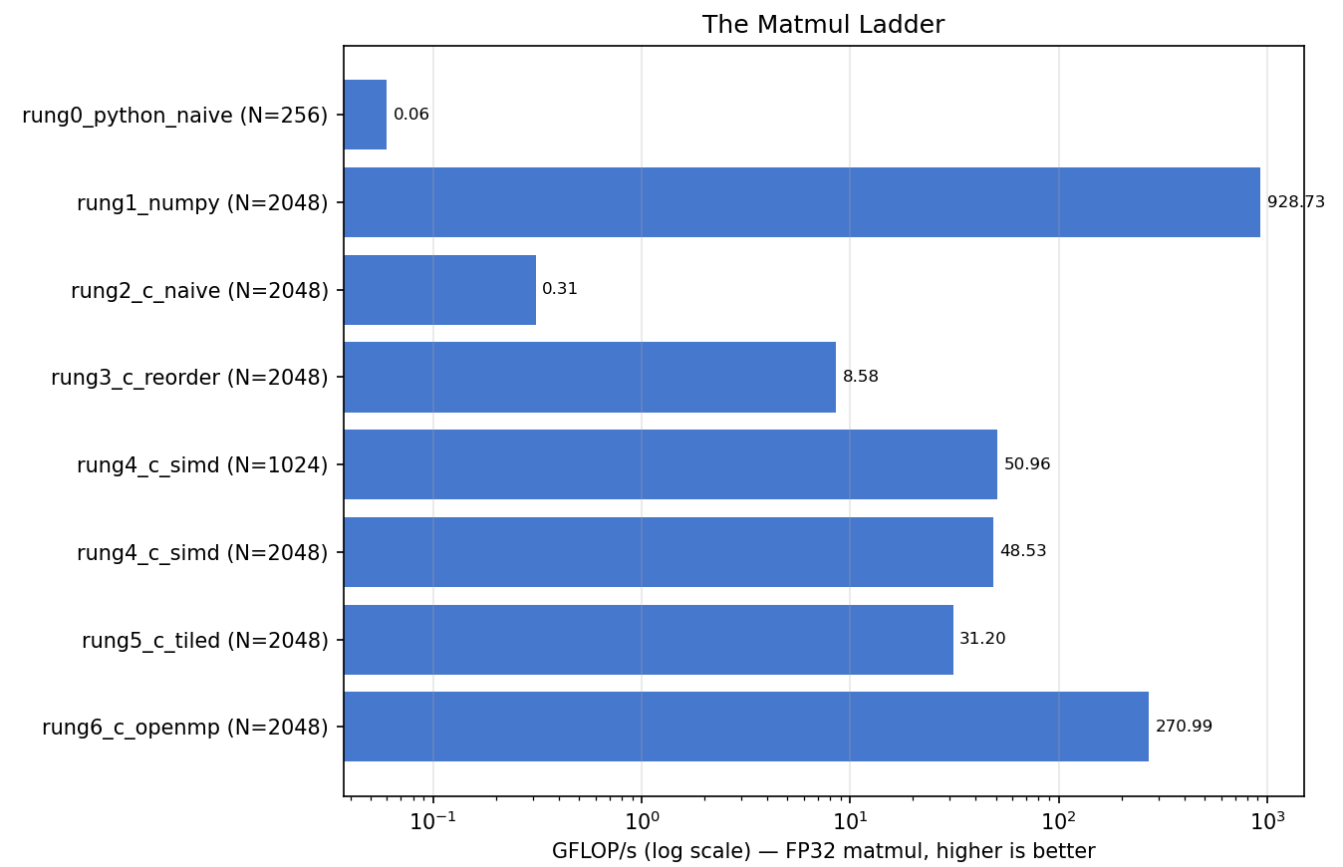


DS635 Lab 1 — The Matmul Ladder

Group

Name	Roll No
Dhruv Parmar	202518030
Mahak Khurdia	202518039

1. Machine identity and `ladder.png`



2. The five filled tables

Table 1 — The ladder (Part 1)

Rung	My GFLOP/s	Lecture laptop	My jump vs previous rung
Naive Python (N=256)	0.06	0.05	—
NumPy (N=2048)	928.73	249	×15,479
Naive C (N=2048)	0.31	0.37	÷2,996
Loop reorder (N=2048)	8.58	3.5	×27.7
SIMD (N=1024)	50.96	31.5	×5.94
SIMD (N=2048)	48.53	14.2	×0.95
Tiled (N=2048)	31.20	30.8	×0.64
OpenMP (N=2048)	270.99	106	×8.69

Table 2 — Cache detective (Part 2)

	Predicted	Measured	Ratio
Naive D1 (lab command, 64 B)	1,073,741,824	1,076,182,547	1.002
Reordered D1 (lab command, 64 B)	67,108,864	67,470,868	1.005
Naive D1 (real geometry, 128 B)	1,073,741,824	1,076,032,000	1.002
Reordered D1 (real geometry, 128 B)	33,554,432	33,720,576	1.005
Naive LLd (real geometry)	131,072 (compulsory)	132,248	1.009
Reordered LLd (real geometry)	131,072 (compulsory)	132,247	1.009

Table 3 — Memory wall (Part 3)

N	B = 4N ²	vs 16 MB L2	GFLOP/s	Δ
512	1.0 MB	fits easily	45.34	—
1024	4.0 MB	fits	49.70	+9.6%
1536	9.0 MB	fits	48.52	-2.4%
2048	16.0 MB	exactly at capacity	49.07	+1.1%
3072	37.7 MB	2.4× over	45.79	-6.7%
4096	67.1 MB	4.2× over	35.21	-23.1% ← cliff

Table 4 — Tile-size sweep (Part 4a), N=2048

T	3·T ² ·4 bytes	Fits in...	GFLOP/s
32	12 KB	L1d (128 KB)	4.33
64	48 KB	L1d	15.44
128	192 KB	L2 (16 MB)	30.95
256	768 KB	L2	36.02 ← best of prescribed set
(extra) 16	3 KB	L1d	7.22
(extra) 512	3 MB	L2	37.43 ← best overall
(reference) untiled rung4_c_simd	—	—	49.07

Table 4b (for Q4.2) — guard for (int j = jj; j < jj + T && j < n; j++), T=256, N=2048:

	GFLOP/s	fmla v<N>.4s	any v<N>.4s	ymm
Without guard	35.61	16	50	0
With guard	7.17	0	32	0
Cost	÷4.97	−100%	−36%	—

Table 5 — Thread scaling (Part 5), N=2048, T=128, OMP_PLACES=cores

Threads	GFLOP/s	Speedup
1	23.30	1.00×
2	44.50	1.91×
4	89.40	3.84×
6	113.99	4.89×
8	158.99	6.82×
12	161.95	6.95× ← flat
16	274.52	11.78×
18	272.71	11.70×

Threads	1	6	8	12	16	18
GFLOP/s	30.72	170.52	205.70	277.07	281.25	280.83
Speedup	1.00×	5.55×	6.70×	9.02×	9.16×	9.14×

3. Answers to Q1.1 – Q5.1

Q1.1 — Which single rung gave the biggest multiplicative jump on your machine? Was it the same rung as on the lecture laptop?

The loop reorder, at 27.7× (0.31 to 8.58 GFLOP/s). SIMD only managed 5.94× and OpenMP 8.69×. It was the same rung on the lecture laptop, but their jump was 9.46×, so mine is about 2.9 times larger. That comes down to my 128 B cache line: in the naive loop each B access uses 4 of the 128 bytes fetched, roughly 3%, where on their 64 B line it was 6.25%, so making the access stride-1 recovers twice as much here. Cachegrind agrees, D1 misses drop 31.9×.

Q1.2 — Pick the rung where your number differs most (as a ratio) from the lecture laptop's, and explain the difference using a hardware fact about your machine.

The biggest gap is SIMD at $N=2048$, 48.53 against their 14.2, a factor of 3.42. It only appears at that size, since at $N=1024$ I am just 1.62 $\times$ ahead. The reason is that their laptop has a 16 MB L3 and mine has no L3 at all. B is exactly 16 MB at $N=2048$, so it thrashes their L3 and they fall from 31.5 to 14.2, while my 16 MB L2 belongs to the 6-core Super cluster alone and I only lose 4.8%. It is not that my CPU is faster, since I am getting that with 4 SIMD lanes against their 8.

Q2.1 — The naive run's read miss rate comes out to almost exactly 50%. The inner loop makes exactly two reads per iteration. Which one misses, which one hits, and why?

B misses and A hits. A walks along a row with stride 1, and the whole row is only 4 KB, so it sits in the 128 KB L1d and almost always hits. B walks down a column with a stride of 4 KB, far larger than the 128 B line, so every access needs a fresh line, and reusing it would mean keeping all 4 MB of B in L1d. That works out to one miss per two reads, which is the 50.0% cachegrind reports. A's own misses come to 2,440,723, only 0.23% of its reads.

Q2.2 — L1d misses are nearly identical for both versions, even though one is 9 $\times$ faster. What are these misses (name the category), and what does their equality tell you about where in the hierarchy the 9 $\times$ was won?

They are compulsory misses. A, B, C_ref and C add up to 16 MB, which is 131,072 lines of 128 B that have to be fetched from DRAM once whatever the loop order is, and I measured 132,248. One caveat: this only holds with my real cache sizes. Under cachegrind's default 256 KB LL the two differ by 16 $\times$, but forcing a 16 MB LLC makes them match to a single miss (132,248 against 132,247). Since both versions move the same bytes across the DRAM bus, none of the speedup came from there. It was won between L1 and L2, where D1 misses fell 31.9 $\times$.

Q3.1 — Where did your cliff land relative to your predicted N^* ? If they disagree, consider: is L3 shared or per-core-complex on your CPU? Does the working set include more than just B?

I predicted $N^* = 2048$, but the cliff only starts between 3072 and 4096, and it is far gentler than theirs (−23% against −55%). Even at $N=3072$, where B is 2.4 times bigger than L2, I still get 45.79. Three things explain it. My L2 is per-cluster rather than shared, and it is not the last level either, since Apple's SLC sits behind it and its size is not reported anywhere, so I could not put it into N^* . The loop also only needs one row of B and one of C at a time, which is a sequential stream the prefetcher handles well, so running out of cache costs bandwidth rather than latency. And the arithmetic intensity is fixed at 0.5 FLOP/byte, so GFLOP/s is simply half the delivered bandwidth:

N	traffic $4N^3$	time	delivered BW	$0.5 \times \text{BW}$	measured
2048	34.4 GB	0.350 s	98.2 GB/s	49.1	49.07
3072	116 GB	1.266 s	91.6 GB/s	45.8	45.79
4096	275 GB	3.904 s	70.5 GB/s	35.2	35.21

Every point matches, so the wall is really single-core bandwidth falling from about 98 to 70 GB/s, not a capacity edge.

Q3.2 — Same binary, same flags, same algorithm at every N — in one sentence, what changed across the cliff?

Only where B's bytes came from: on-package cache at about 98 GB/s up to $N \approx 3072$, then DRAM at about 70 GB/s at 4096, and at a fixed 0.5 FLOP/byte that 1.4 $\times$ drop in bandwidth is the 1.4 $\times$ drop in throughput.

Q4.1 — Explain your winning T from your cache sizes. Why does T=256 (or your largest losing T) fall off, even though bigger tiles mean more reuse per byte?

My prediction was wrong and nothing falls off. T=256 wins the prescribed set at 36.02, T=512 is still improving at 37.43, and tiling never beats the untilted version at 49.07, so even my best tile is 24% slower. Large tiles cannot fall off here because even T=512 needs only 3 MB, about 19% of my 16 MB L2, and I would need roughly T=1180 before capacity mattered. Small tiles are what hurt: T=16 leaves only 4 NEON iterations in the inner loop, far too few to cover the unrolling and the five outer loops, and tile rows sit 8 KB apart so they collide in the same cache sets. That is probably why T=32 comes out worse than T=16, although I did not confirm it. The real reason tiling does nothing is Part 3, since rung 4 was never memory-bound at N=2048 and there was no problem for it to fix.

Q4.2 — Report GFLOP/s and the instruction count with and without the guard. The guard is semantically harmless here ($N \% T == 0$, so $j < n$ is always true). Why does the compiler give up vectorizing anyway — what would it have to prove, and can it?

35.61 drops to 7.17 GFLOP/s, about 5×, and the vector `fm1a` count goes from 16 to 0. So the guard is not costing a compare, it stops the loop being vectorized at all, and losing 4 lanes plus the unrolling is roughly that factor. GCC gives the reason under `-fopt-info-vec-all`: `not vectorized: number of iterations cannot be computed`. To keep vectorizing it would have to prove $jj + T \leq n$ for every tile, so the trip count stays the constant 256. It cannot, because that only follows from $n \% T == 0$, which requires both `jj` and `n` to be multiples of T, and GCC tracks value ranges rather than divisibility. The compiler optimizes what it can prove, not what happens to be true.

Q5.1 — Name the three shared resources from lecture that eat the missing speedup, and point at the evidence for at least one of them in your table.

Memory bandwidth is the main one: at 16 threads the kernel needs about 549 GB/s and scaling simply stops, with 18 threads giving nothing over 16 (274.52 against 272.71). Shared cache capacity is second, and the clearest evidence is the first sag at 4→6 threads, where efficiency falls from 96% to 82%, exactly where the 6-core Super cluster and its 16 MB L2 fill up. Third is core heterogeneity and clock, since 6 cores are Super and 12 are Performance, and `OMP_PLACES=cores` cannot tell them apart because `lscpu` reports one flat 18-core cluster.

There is a fourth thing that actually shapes my table, which is load imbalance. At N=2048 with T=128 the parallel loop has only 16 iterations, so static scheduling over 12 threads leaves some with 2 chunks and caps the useful parallelism at 8. That predicts the 8-thread result of 158.99, against the 161.95 I measured. At 16 threads everyone gets one chunk and it jumps to 11.78×. Running the same sweep at N=3072, which has 24 tiles, removes the effect and 12 threads scale cleanly to 9.02×.

Stretch — Break the harness on purpose (`-ffast-math`)

I predicted that dropping `-ffast-math` would cost most of the SIMD speedup. It cost nothing. Absolute values sit ~1.6× under §2 because the machine was under load, so only the ratios here are meaningful.

Build (N=2048, 3 reps)	GFLOP/s	vector fm1a v.4s
rung4 SIMD, <code>-ffast-math</code>	32.01	90
rung4 SIMD, strict IEEE-754	31.99	90
rung3 scalar reorder, untilted	5.49	0
rung3 scalar reorder, tiled T=128	4.79	0

Rung 3 had already removed the reduction: $C[i*n+j] += a * B[k*n+j]$ is elementwise across j , so each lane accumulates into a different element of C , the summation order never changes, and vectorizing is bit-exact under IEEE-754 without any permission. The rung that does need reassociation is the naive i - j - k loop, where $s += \dots$ is a real reduction and `fmla` goes $14 \rightarrow 0$ without the flag — yet allowing it there loses (0.45 vs 0.58 GFLOP/s), since that loop is limited by cache misses, not arithmetic.

Tiling with scalar flags confirms Part 4: 4.79 against 5.49 GFLOP/s until, a 13% loss. Tiling only pays once the compute roof is high enough to hit the memory wall.